

Implement FPGA and Test a Strong PUF (Arbiter)

EEE5716 Introduction to Hardware Security and Trust Final Project

Arif Hussain

Electrical and Computer Engineering

University of Florida

Gainesville, United States

arifhussain@ufl.edu

Abstract -- This work presents the design, implementation, and evaluation of a delay-based Physical Unclonable Function (PUF) on an FPGA using symmetric racing delay lines and an arbiter. Motivated by the need for secure, hardware-rooted key generation and device authentication, the project focuses on constructing reliably matched delay paths, ensuring symmetry through direct LUT instantiation, and analyzing the resulting Challenge-Response Pairs (CRPs). A comprehensive workflow spanning Verilog development, hardware testing, large-scale simulation, and Python-based post-processing was used to characterize the PUF across reliability, uniqueness, and uniformity metrics. Results show perfect reliability and strong uniformity, though uniqueness fell short of the ideal Hamming-distance target, indicating the need for further parameter tuning and expanded simulations. Overall, the implemented PUF demonstrates viable performance for secure FPGA applications, with future work aimed at improving uniqueness and scaling the design to generate larger CRP sets.

Index Terms -- FPGA, PUF, Arbiter, RTL, Hardware Security

I. INTRODUCTION

Field-programmable gate arrays (FPGAs) are configurable integrated circuits that can be programmed repeatably after fabrication. Because they are inherently more flexible than application-specific integrated circuits (ASICs), they are used in a wide range of applications, from transportation to communications [1]. Their inclusion in these applications means that they require a certain degree of security. Processes such as encryption are necessary to maintain IP security and prevent the exposure of various secrets.

In hardware security, a secret is any information that only select individuals are supposed to have. One example of this concept is the use of public and private keys for encrypting and decrypting messages from ciphertext to plaintext. When sharing information amongst individuals, the secret keys can be kept confidential, but when the keys are stored on the hardware, a malicious third party can extract them. Because the keys cannot be changed easily once the hardware has been deployed, this creates a serious security concern. This can be addressed by using Physical Unclonable Functions (PUFs).

PUFs are physical objects that cannot be cloned or modeled using software. They are a natural source of entropy that can create a truly random function. In the case of FPGAs, they leverage the natural oscillations that come from current flowing throughout the physical structure of the circuit.

Creating a PUF is an essential part of hardware security as it allows for key generation and device authentication [2]. The responses returned as part of a challenge are considered unique to every device because PUFs rely on the random nature of manufacturing to determine the response. The approach pursued in this project involves creating PUFs using delay times and an arbiter to compare different routes. One of the key challenges is creating symmetric paths that the arbiter compares so the results for each device will be unique and random. Asymmetric paths can cause predictability and result in a vulnerable PUF design. The main goals of this project are to create an implementation of a delay-based strong PUF and then evaluate it based on reliability, uniqueness, and uniformity.

II. PUF IMPLEMENTATIONS

A. Architecture

To create a strong PUF on an FPGA, two main components are needed. The first is the medium used for the PUF. When using FPGAs, one of the most effective approaches is racing delay lines. In the hardware code, two seemingly identical lines are created, both serving the same function and algorithm. When bits are given to the lines as a challenge, both should finish at the same time, but this is not the case since the latency along these lines can differ. Assuming both lines have the same length and the same operation, the only difference will be how long it takes the signal to reach the end. This is influenced by the physical makeup of the circuit, such as imperfections in the silicon.

Specifically, the system uses 32 cascaded switch blocks (stages) that create two parallel race tracks for the signal. Multiplexers were implemented using Look-Up Tables (LUTs) to create the switch blocks. Each path was instantiated directly to ensure correct behavior, as the racing paths did not operate correctly with standard behavioral Verilog.

The second component is an arbiter. The arbiter takes the two racing lines and determines which finishes first. Once the lines finish racing, the arbiter outputs a single bit (0 or 1) based on which line finishes first. The PUF can be used repeatedly with different challenge bits, and the arbiter assembles the individual bits into a multiple-bit string. These strings can be used for encryption and device keys. For this project, a D-Flip-Flop was used as the arbiter, which captured the first finisher of the delay paths to determine the result. The final system is a 32-bit Strong Arbiter PUF targeting the Real Digital Urbana FPGA (Spartan-7 XC7S50).

Several considerations arise when creating the architecture of the racing lines. The first is symmetry: when a design is placed on an FPGA, the two racing lines can be asymmetric, meaning that although they are identical in code, the synthesizer, placement, and routing may not produce symmetric physical paths. This can cause one line to always finish first. A second consideration is the outside environment: when temperature and supply voltage are affected, the PUF must still produce the same responses. The final consideration is uniqueness: if the same code is placed on the same type of FPGA, the responses should still differ between individual chips, as the PUF should rely solely on the physical composition of each device.

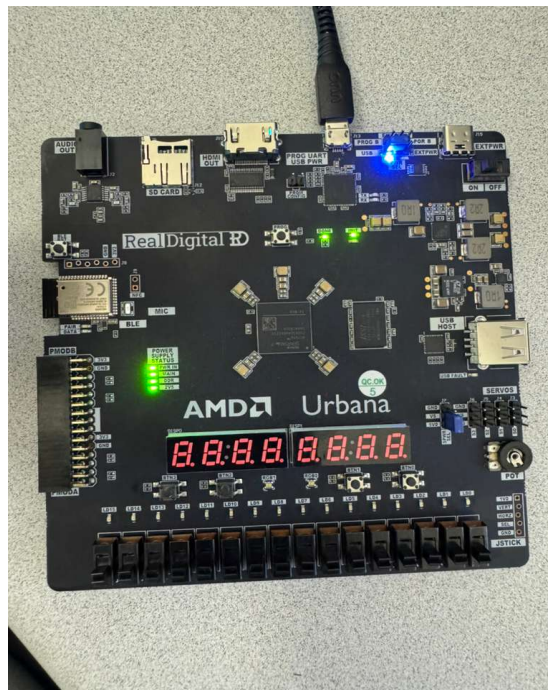


Fig. 1. Real Digital Urbana FPGA board (Spartan-7 XC7S50) with all switches in the down position.

B. Operating Principle

The principle behind the PUF is that if two identical circuits are created and given a certain input, one will produce an output before the other. While they are the same circuit, the physical differences between the components cause timing

variations. This is driven by factors including temperature differences during manufacturing, inconsistency in materials, and flaws in the completed circuitry.

This principle is leveraged when a PUF is created. The type of PUF implemented here is delay-based, using two identical lines so that when a set of challenge bits is given to both lines, one finishes first as determined by the arbiter. For every set of challenge bits, a set of response bits is received, resulting in Challenge-Response Pairs (CRPs) that can be used for encryption and device authentication keys.

Three key principles govern PUF design. First, reliability: a PUF should always return the same response for the same challenge bits, which is essential for device authentication. Second, uniqueness: responses should differ across chips, as identical responses from different FPGAs would indicate the PUF depends on the design rather than physical variations. Third, uniformity: a PUF that is not uniform in its response distribution (i.e., biased toward 0s or 1s) is more susceptible to machine learning attacks because the responses can be modeled.

C. Evaluation Experiments

The project workflow consisted of four major steps. First, a project was created in Vivado with the Register-Transfer Level (RTL) sources targeting the Urbana board constraints. Second, hardware testing was completed to ensure the PUF design could be correctly loaded onto the physical Urbana board. Third, large-scale simulations were run by feeding challenge bits into the PUF and collecting responses for CRP generation. Finally, the CRPs were analyzed and processed to determine Reliability, Uniqueness, and Uniformity.

The PUF code was developed in Verilog and evaluated using Vivado 2024.2. Once uploaded to the FPGA after synthesis, placement, and routing, experiments could be run. Verilog testbenches were set up to read various challenges and store the responses. Python scripts were then used to analyze and calculate the metrics.

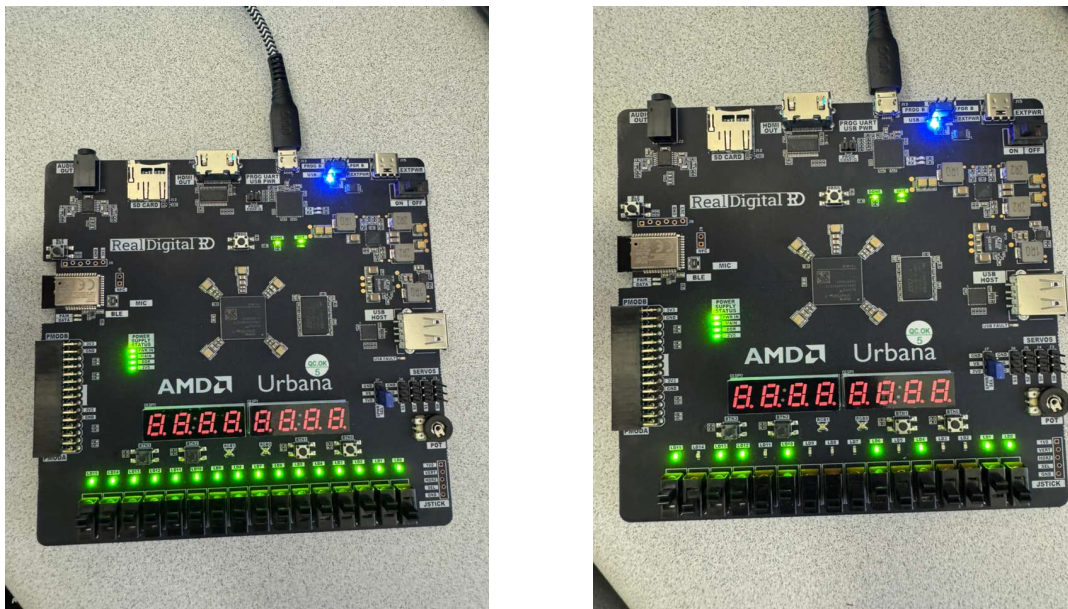


Fig. 2. Hardware testing on the Urbana FPGA: all switches up (left) and a random challenge pattern (right), demonstrating PUF response output on LED0.

The code creates a chain of switches. The number of stages (32 in this case), along with a random seed, can be used to create a unique delay configuration. The arbiter determines the output for each challenge, completing a CRP. With the basic PUF delay lines and arbiter setup, the PUF was used with testbenches to create a large quantity of CRPs. Different seeds and stages were tested to find the optimal configuration, and the results were analyzed using Python scripts.

Reliability was checked by determining the Bit Error Rate (BER) across challenges. Uniqueness was determined using the Hamming distance between response pairs from different simulated chips, where a greater Hamming distance is desired. Uniformity was evaluated by counting the fraction of 1s in the response bits, with an ideal target of 0.50.

D. Post-Processing Algorithms

With a working PUF and defined metrics, simulations were run and results stored in files. Post-processing was achieved through Python scripts. Each metric had a dedicated analysis script (`analyze_reliability.py`, `analyze_uniqueness.py`, `analyze_uniformity.py`), and a top-level script (`run_all_analyses.py`) could run all processing to determine all metrics with one command. This allowed running different variations of the PUF to find the best parameters and create an optimal delay-based PUF.

E. Hardware Mapping

An important early consideration was how the PUF would be instantiated in the FPGA and ensuring the two racing paths were symmetric. Without symmetric paths, the design would not rely on hardware differences but rather just path length. The approach taken did not use standard behavioral Verilog logic. Instead, LUT primitives were instantiated directly so the delay paths could be controlled precisely. This produced symmetric delay lines that could be directly controlled, yielding a working PUF and arbiter ready for testing.

Due to the scope of this project, additional testing such as power supply and temperature variation effects on results was not performed. This is an area for future work to ensure proper PUF operation and adaptability to other applications.

F. Simulation Environment

Two testbenches were developed in Vivado for data collection. The single-chip testbench (arbitrPUF_single_tb) feeds challenges into one PUF instance to collect CRPs for reliability and uniformity analysis. The multi-chip testbench (arbitrPUF_multi_tb) instantiates multiple PUF instances with different seeds to simulate inter-chip variation for uniqueness evaluation.

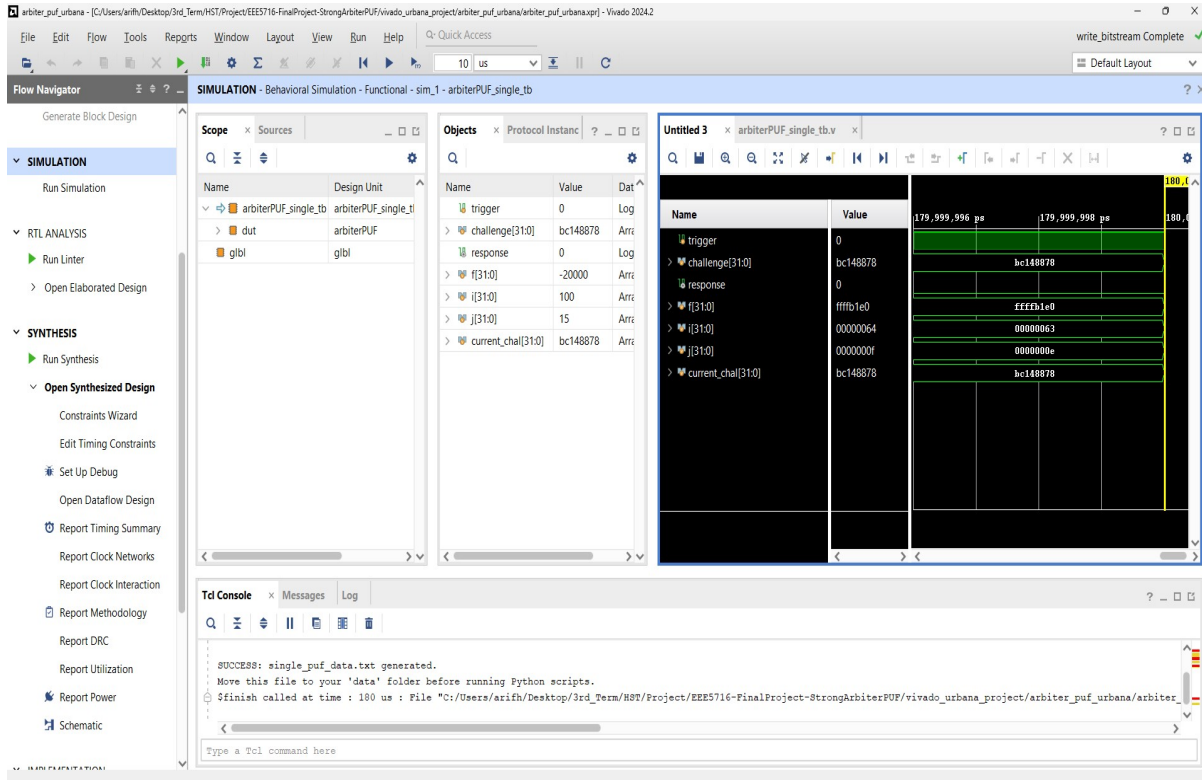


Fig. 3. Vivado behavioral simulation of the single-chip testbench (arbitrPUF_single_tb) generating single_puf_data.txt.

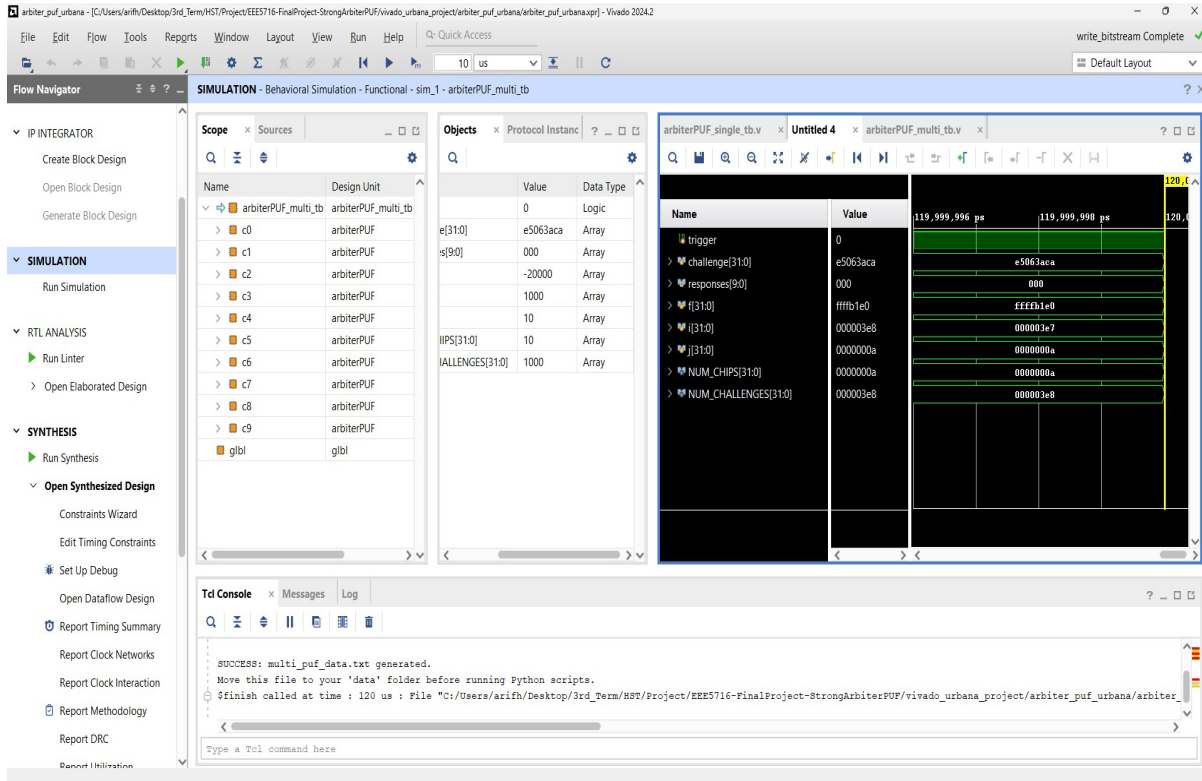


Fig. 4. Vivado behavioral simulation of the multi-chip testbench (arbitrerPUF_multi_tb) generating multi_puf_data.txt for uniqueness analysis.

III. RESULTS AND EFFECTIVENESS

Given the tests and experiments performed, the following results were obtained for the three target metrics.

A. Reliability

Reliability is critical for a PUF, especially when device authentication depends on it. The same response is needed every time to ensure authentication. From the design and the tests run, the results showed a perfect score. The PUF was completely reliable with zero Bit Error Rate (BER = 0.0000), meaning it produced identical responses for repeated challenges across all test runs. This confirms that the PUF can be used dependably for device authentication.

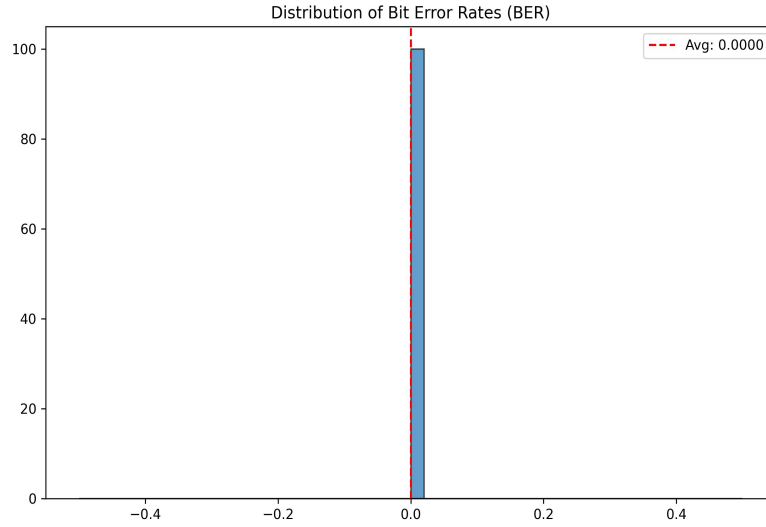


Fig. 5. Distribution of Bit Error Rates (BER) showing perfect reliability with Avg. BER = 0.0000.

B. Uniqueness

Aside from reliability, it is important that the results from a challenge do not produce the same response on multiple chips. This would indicate that the PUF design depends on the architecture rather than the physical board itself. The design resulted in an average Hamming distance of 0.192667. The ideal target was 0.50, so in terms of uniqueness, additional work and simulations are needed to optimize parameters. The lower-than-ideal uniqueness is attributed to the lack of manual routing symmetry (which is very difficult to achieve on FPGA) and having only one real FPGA available for hardware variation testing.

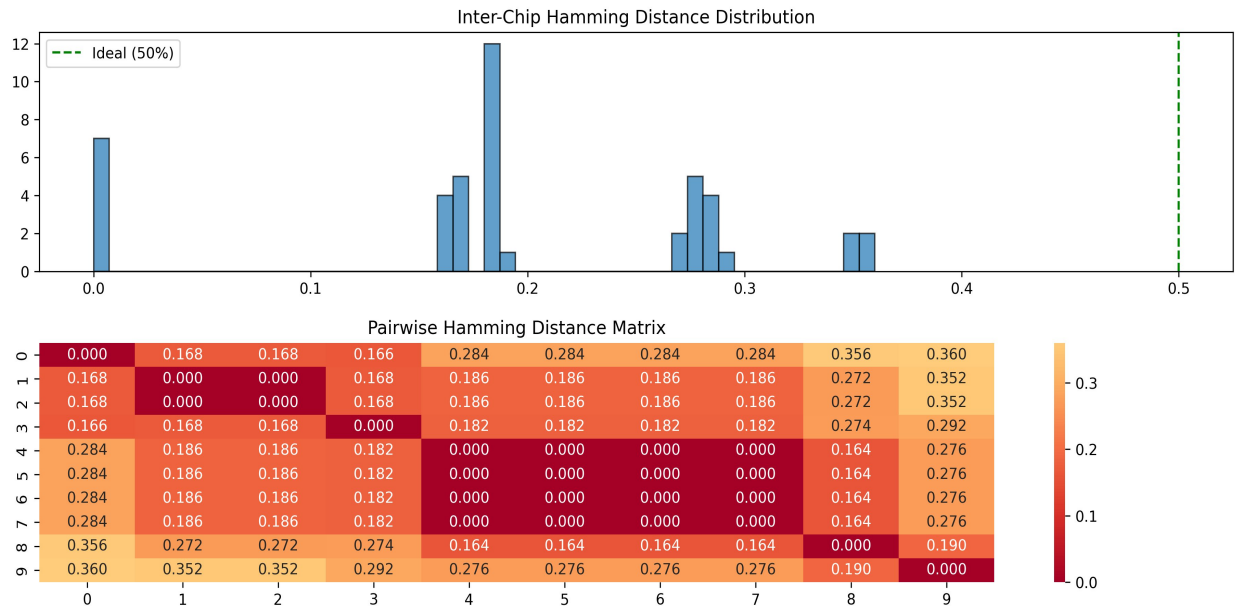


Fig. 6. Inter-chip Hamming distance distribution (top) and pairwise Hamming distance matrix (bottom) across 10 simulated chips. Average uniqueness = 19.27% vs. the ideal 50%.

C. Uniformity

A PUF that is uniform in its responses to challenges is necessary. PUFs that are not uniform and are biased in their responses are more susceptible to machine learning attacks because the responses can be modeled. When running the tests and testbenches, the PUF returned results that were almost an even split between 0 and 1. The Average Fraction of 1s was 0.4782, which is very close to the ideal 0.50 and demonstrates sufficient uniformity.

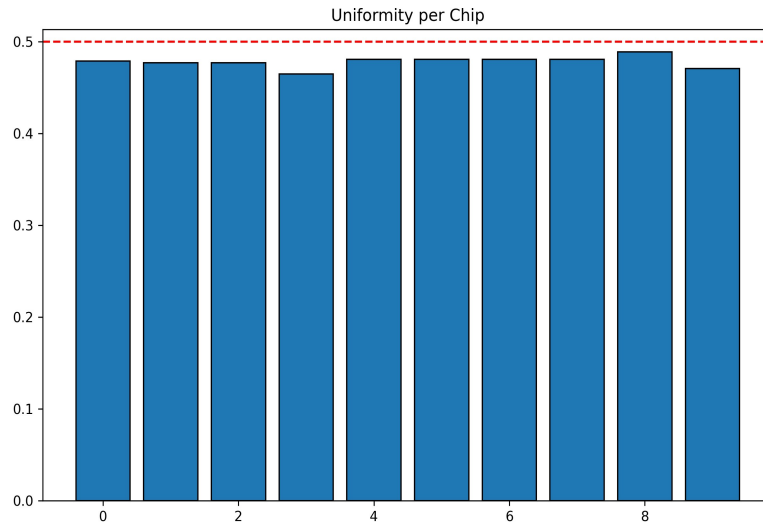


Fig. 7. Uniformity per chip showing the fraction of 1s near the ideal 0.50 threshold (dashed red line). Average = 0.4782.

Metric	Result	Ideal	Assessment
Reliability (BER)	0.0000	0.0000	Perfect
Uniqueness (Hamming Dist.)	0.1927	0.5000	Needs improvement
Uniformity (Fraction of 1s)	0.4782	0.5000	Strong

Table I. Summary of PUF evaluation metrics.

IV. RESOURCE UTILIZATION

The design was synthesized using Vivado 2024.2 targeting the Spartan-7 XC7S50 (xc7s50csga324-1). The PUF implementation is extremely lightweight, consuming minimal FPGA resources. The utilization report from synthesis is summarized below.

Resource	Used	Available	Utilization
Slice LUTs	53	32,600	0.16%
LUT as Logic	53	32,600	0.16%
Slice Registers	1	65,200	<0.01%
Register as Flip Flop	1	65,200	<0.01%
Bonded IOB	33	210	15.71%

Block RAM	0	75	0.00%
DSPs	0	120	0.00%

Table II. FPGA resource utilization for the 32-bit Strong Arbiter PUF on the Spartan-7 XC7S50.

The design uses only 53 LUTs (64 LUT3 primitives for the switch stages and 21 LUT1 primitives for routing) and a single flip-flop for the arbiter. The 33 bonded IOBs account for the challenge input switches, clock, trigger, and LED outputs. No Block RAM or DSP resources are consumed, confirming the PUF is a purely combinational design with minimal overhead.

Primitive	Count	Category
LUT3	64	LUT
LUT1	21	LUT
IBUF	17	IO
OBUF	16	IO
FDRE	1	Flip Flop & Latch

Table III. Primitive breakdown of the synthesized PUF design.

V. CONCLUSION

PUFs are useful for creating secure secrets on FPGAs. By leveraging the unique properties of the PUF, unique challenge-response pairs can be generated, ensuring that the data on the FPGA can remain a secret. To this end, this project has successfully designed, implemented, and tested a 32-bit Strong Arbiter PUF on an FPGA.

The PUF achieved passing results in most of the main categories. For reliability, the PUF performed perfectly, giving consistent responses throughout testing with zero bit error rate. The uniformity of the PUF was also strong, achieving a near-even split (0.4782) when testing PUF responses.

The area requiring improvement is uniqueness. During testing, the PUF could not achieve a sufficient Hamming distance (0.1927 vs. the ideal 0.50). This is likely due to limited routing control and having only a single physical FPGA for testing.

Future work in this project would include performing more simulations and testing to optimize the parameters for uniqueness, further optimizing uniformity, testing under temperature and voltage variation conditions, and adding more delay lines to increase the number of CRPs for the PUF.

REFERENCES

- [1] M. Majzoobi, A. Kharaya, F. Koushanfar and S. Devadas, "Automated design, implementation, and evaluation of arbiter-based PUF on FPGA using programmable delay lines," 2014.
- [2] A. Maiti et al., *A systematic method to evaluate and compare the performance of physical unclonable functions*, In Embedded Systems Design with FPGAs, P. Athanas, D. Pnevmatikatos, and N. Sklavos (Eds.). Springer, New York, pp. 245-267.